Experiment 8: Use Postman to test sample API

1. Objective

- Understand how to test REST APIs using Postman by sending different types of HTTP requests.
  - Get
  - Post
  - Put
  - Delete

2. Prerequisites

- postman
- API to be tested
  - Create a simple node.js API / use some free fake API

3. Test API

Get:
URL: https://jsonplaceholder.typicode.com/posts
Post:
https://jsonplaceholder.typicode.com/posts

```
{
  "title": "Test Post",
  "body": "This is a sample post",
  "userId": 1
}
```

Put:
https://jsonplaceholder.typicode.com/posts/1

```
{
  "id": 1,
  "title": "Updated Title",
  "body": "Updated content",
  "userId": 1
}
```

Delete:
https://jsonplaceholder.typicode.com/posts/1

Experiment 8: Use Postman to test sample API

4. Create node.js API

Step 1: Create Project

```
mkdir simple-api
cd simple-api
npm init -y
npm install express
```

Step 2: Create server.js file

```
const express = require('express');
const app = express();
app.use(express.json());

// In-memory data (acts like database)
let students = [
  { id: 1, name: "Viraj", branch: "CE" },
  { id: 2, name: "Ravi", branch: "CE" }
];

// GET all students
app.get('/students', (req, res) => {
  res.json(students);
});

// GET single student
app.get('/students/:id', (req, res) => {
  const student = students.find(s => s.id == req.params.id);
  res.json(student);
});

// POST - Add student
app.post('/students', (req, res) => {
  const newStudent = {
    id: students.length + 1,
    name: req.body.name,
    branch: req.body.branch
  };
  students.push(newStudent);
  res.status(201).json(newStudent);
```

```
});

// PUT - Update student
app.put('/students/:id', (req, res) => {
  const student = students.find(s => s.id == req.params.id);

  if (student) {
    student.name = req.body.name;
    student.branch = req.body.branch;
    res.json(student);
  } else {
    res.status(404).json({ message: "Student not found" });
  }
});

// DELETE - Remove student
app.delete('/students/:id', (req, res) => {
  students = students.filter(s => s.id != req.params.id);
  res.json({ message: "Student deleted" });
});

// Start server
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

<u>Step 3: Run server</u>

Node server.js

Experiment 8: Use Postman to test sample API

5. Procedure TEST using postman

1. GetAll:

Type: Get
URL : http://localhost:3000/students
Check response

2. Post:

Type: Post
URL: http://localhost:3000/students
Body:

```
{
  "name": "Neha",
  "branch": "CE"
}
```

Check response and Send Get request to see new data inserted

3. Put request:

Type: PUT
URL : http://localhost:3000/students/1
Body:

```
{
  "name": "Viraj Updated",
  "branch": "IT"
}
```

Check response
Send Get request to see updated data

4. Delete request:

Type: DELETE
URL : http://localhost:3000/students/1
Check response
Send Get request to see updated data